Guidelines and template for FAIKR-Mod3 Project Reports

Daniele Tarek Iaisy
Master's Degree in Artificial Intelligence, University of Bologna
danieletarek.iaisy@studio.unibo.it

July 5, 2026

Abstract

Learning the structure of a Bayesian Network, represented as a Directed Acyclic Graph (DAG), is a computationally challenging task due to the combinatorial nature of the search space and strict acyclicity constraints. Continuous numerical optimization has emerged as a promising alternative to traditional combinatorial search methods. This study investigates and compares novel continuous optimization techniques, specifically NOTEARS and DAG-GNN, against classical algorithms (PC, FGES, and Hill Climbing) in terms of structural fidelity and computational efficiency. We evaluated these algorithms on synthetic datasets of varying complexities (ranging from 5 to 37 nodes) and a real-world biological dataset. Our results reveal a distinct trade-off between the approaches: continuous methods, yield lower Structural Hamming Distances (SHD) as network complexity grows and scale well with the number of variables. In contrast, classical algorithms demonstrate superior scalability concerning dataset size but suffer from computational bottlenecks on larger networks. Notably, despite its architectural suitability for graph-structured data, DAG-GNN underperformed compared to the simpler NOTEARS baseline while requiring significantly higher computational resources.

Introduction

Domain

Learning a Bayesian Network from a data can be a difficult task. The space of DAGs over a set of nodes is a combinatorial space and traditional approaches have a super-exponential worst case complexity, mainly owed to the aciclicity constraint to be enforced on the learned DAG. Continuous numerical optimization techniques can be a viable option to speed up the search.

Aim

The main objective of this study is to understand how novel techniques based on numerical optimization work compared against more traiditional approaches, both in terms of the fidelity of the learned network and in terms of resources necessary to run each training algorithm.
To do so, we investigate the NOTEARS algorithm (TODO reference) and the DAG-GNN algorithm (TODO) compared to the classical algorithms PC, FGES, and hill climbing. We selected NOTEARS because it is the pioneering work that first transformed DAG structure learning from an intractable combinatorial search into a purely continuous optimization problem. It works by introducing a smooth, exact algebraic characterization of the acyclicity constraint which allows the network structure of a linear structural equation model to be efficiently estimated using standard black-box numerical solvers. DAG-GNN was subsequently chosen as a natural extension of this continuous framework, leveraging the representational power of Graph Neural Networks (GNNs). It works by employing a GNN-parameterized Variational Autoencoder (VAE) to capture the underlying data distribution alongside a polynomial variant of the continuous acyclicity constraint. We included DAG-GNN to investigate another approach in this space on top of NOTEARS and specifically because the use of GNN-parametrized VAE seems a natural fit for the task of DAG learning.

Method

What we want to test is structure learning, how good an algorithm can recover the original Bayesian network representing the probability distribution from data sampled from said distribution. To do so we need a ground truth network to compare against the one constructed from our learning algorithm. The blearn library (todo reference https://www.bnlearn.com/bnrepository/) contains a repository of prebuilt Bayesian networks that can be used to generate a synthetic dataset, with DAGs ranging from just 5 variables to 1041. To avoid only using synthetic data we also tested on the protein folding dataset introduced in (TODO cite Sachs et al.) which contains a DAG with 11 nodes and a dataset of 11672 real samples. The full experiment covers 4 networks with node count ranging from 5 to 37.
For each experiment, we report the performance metric of the learned graph, main one being the SHD, and the execution time of the algorithm to learn the output graph.

Results

NOTEARS appears to be the winner as it usually recovered the best DAGs among all of the tested algorithms. Despite the expectation of DAG-GNN being a good fit for structure learning, it performed worse than NOTEARS and some-

times worse than classical methods, while being much more costly to run. These approaches are not particularly affected by an increase in variable count, and scale well along this dimension, while being much more sensitive to sample count in the dataset used, something that classical methods do not suffer particularly.

Model

Many of the Bayesian network tested for this study come from the bnlearn repository of networks. Specifically, we used the CANCER composed of 5 variables, the CHILD composed of 20 variables, and the ALARM one composed of 37 variables. All of these network include discrete variables only. For more details on the networks we refer the reader to the website: TODO.
In addition to these discrete benchmarks, we evaluate algorithms on a real-world continuous dataset introduced by Sachs et al. (**?**), derived from single-cell flow cytometry measurements. The ground truth network consists of 11 continuous variables representing signaling proteins, connected by 17 high-confidence directed arcs.

Analysis

Experimental setup

The objective is to perform analysis on what algorithm performs best in recoverint the original DAG, so for each dataset we run the algorithm and check how good the learned DAG is. Each algorithm has its own hyper-parameters and a optimal combination of hyper-parameter on one dataset might not be optimal for a different one. Because of this we run a hyper-parameter grid search over each dataset for each algorithm first to derive good hyper-parameter values for each dataset-algorithm combination.
Then, we execute the algorithm on each dataset and compare the learned DAG against the ground-truth one computing graph reconstruction metrics, along with execution time of the algorithm. The most important metric is SHD (Structural Hamming Distance), which measures how many edge addition, deletion, or edge inversion needs to be performed on a graph to transform it into the other (lower means better).
To investigate the performances at varying scales we selected the CANCER (5 nodes), CHILD (20 nodes), and ALARM (37 nodes) networks from the bnlearn repository and generated a synthetic dataset of 1000 samples for these. We also used the (TODO cite Sachs et al) dataset, containing 11 nodes and 11672 real samples. We also note that in the preliminary phase the BARLEY graph from the bnlearn repository was tested, but ultimately discarded because all of the classical algorithms took extremely long to train on it.

Results

On very small datasets NOTEARS and DAG-GNN do not appear to be helpful compared to classical algorithms as they both recover DAGs with worse SHD than other methods achieved (NOTEARS matches Hill Climbining performance on CANCER, which is the worse performant of classical approaches, DAG-GNN does worse). As the number of nodes increases, NOTEARS starts showing an edge, achieve lower SHD than any other method on both the CHILD dataset and the Sachs et al. one. Notably, DAG-GNN does somewhat better than classical algorithm in these, but does not beat the simpler and faster NOTEARS.
Other than how good of a graph is reconstructed, it's interesting to investigate how long it takes for the DAG to be learned. We observe that NOTEARS and DAG-GNN scale much better as the number of variables grows: these algorithms took roughly the same amount of time across all synthetic datasets, while FGES struggled enough on the ALARM one to be removed from that experiment and both FGES and PC took prohibitively long on the BARLEY dataset (48 nodes) leading to its removal from the experiment. On the other hand, their scaling w.r.t. the dataset size is much less favourable: Hill climbing, PC, and FGES all take roughly the same amount of time on the Sachs et al. dataset that it took them on the CHILD one, despite having roughly 10 times more samples; NOTEARS instead took roughly 20 times more time to terminate and DAG-GNN took 16 times. Table 1 contains the performance of each algorithm on each dataset, along with averaged metrics across all datasets.

DAG-GNN was tested because it seemed a natural fit to the problem: we are trying to learn a DAG and its architechture uses a Graph Neural Network under the hood. Despite this fact in none of the tested networks it was able to outperform NOTEARS, despites still beating classical algorithms except for the simplest case of the CANCER dataset. It is still believable that it could beat NOTEARS as the network becomes bigger and the relationships between variables more complex, but this investigation is left as future work.

Conclusion

In conclusion, this study highlights a distinct trade-off between continuous optimization and classical approaches in Bayesian network structure learning, revealing that continuous methods excel in structural accuracy as network complexity grows, whereas classical algorithms demonstrate superior scalability with respect to dataset size. An interesting and somewhat unexpected finding was the underperformance of DAG-GNN; despite its architecturally natural fit for graph-structured data, it failed to surpass the simpler continuous baseline while incurring significantly higher computational costs. A primary limitation of our current experimental setup is the computational bottleneck experienced by classical algorithms on very large networks, which restricted evaluation to moderately sized graphs, alongside the observed sensitivity of continuous solvers to large sample sizes. Future work should therefore focus on optimizing continuous methods for larger datasets and evaluating these advanced graph-based models on substantially more complex networks to determine if their representational advantages ultimately manifest at a much larger scale.

Links to external resources

Optionally, insert here:

Table 1: Per-dataset and averaged results for each algorithm. *nSHD* = SHD / $|E^*|$ (SHD normalised by the number of true edges), making cross-network averages comparable. *Accuracy* = TP / (TP + FP + FN + rev.). FGES was not run on ALARM; its average excludes that network.

Dataset	Algorithm	Time (s)	SHD	nSHD	Accuracy	Precision	Recall	F1
Sachs	NOTEARS	112.80	20	1.000	0.200	0.385	0.250	0.303
	DAG-GNN	1585.98	22	1.100	0.000	0.000	0.000	0.000
	HC	5.00	37	1.850	0.098	0.121	0.200	0.151
	PC	6.38	30	1.500	0.189	0.250	0.304	0.275
	FGES	2.32	26	1.300	0.235	0.364	0.286	0.320
Cancer	NOTEARS	0.07	4	1.000	0.200	0.500	0.200	0.286
	DAG-GNN	83.63	4	1.000	0.000	0.000	0.000	0.000
	HC	0.19	3	0.750	0.250	0.250	0.250	0.250
	PC	0.05	2	0.500	0.500	0.667	0.500	0.571
	FGES	0.09	1	0.250	0.750	0.750	0.750	0.750
Child	NOTEARS	5.84	22	0.880	0.185	0.333	0.200	0.250
	DAG-GNN	100.21	24	0.960	0.172	0.294	0.200	0.238
	HC	6.02	30	1.200	0.231	0.265	0.360	0.305
	PC	1.73	27	1.080	0.289	0.344	0.379	0.361
	FGES	8.58	27	1.080	0.372	0.421	0.533	0.471
Alarm	NOTEARS	58.17	34	0.739	0.414	0.585	0.522	0.552
	DAG-GNN	306.54	42	0.913	0.276	0.485	0.348	0.405
	HC	27.28	61	1.326	0.282	0.312	0.522	0.390
	PC	4.50	29	0.630	0.517	0.646	0.585	0.614
Average	NOTEARS	44.22	–	0.905	0.250	0.451	0.293	0.348
	DAG-GNN	519.09	–	0.993	0.112	0.195	0.137	0.161
	HC	9.62	–	1.282	0.215	0.237	0.333	0.274
	PC	3.17	–	0.928	0.374	0.477	0.442	0.455
	FGES	3.66	–	0.877	0.452	0.512	0.523	0.514

- a link to your GitHub or any other public repo where one can find your code (only if you did not submit your code on Virtuale)
- a link to your dataset (only if you have used a dataset, for example, for learning network parameters or structure)

Leave this section empty if you have all your resources in Virtuale. **Do not insert code/outputs in this report**.